

DATABRICKS CERTIFIED ASSOCIATE DEVELOPER

BLOQUE 2: Arquitectura de Apache Spark

Ejecución Distribuida, Driver, Executors, DAG & Optimización Interna



Comprensión Profunda del Motor Interno de Spark 3.x

| ¿Por qué Entender la Arquitectura Interna?

Objetivos de Aprendizaje

- ✓ Comprender la arquitectura general y componentes clave.
- ✓ Diferenciar con claridad los roles de Driver y Executors.
- ✓ Entender cómo Spark divide el trabajo (Job → Stage → Task).
- ✓ Dominar conceptos críticos: DAG, Lazy Evaluation y Shuffle.
- ✓ Diagnosticar y monitorear ejecuciones desde la Spark UI.

El Desafío en Desarrollo Real

Muchos desarrolladores escriben código sin entender por qué una consulta tarda horas, por qué se consume tanta memoria o por qué aparecen errores de OOM.

"La mayoría de problemas de rendimiento en Spark NO son problemas de código syntax. Son problemas de arquitectura."

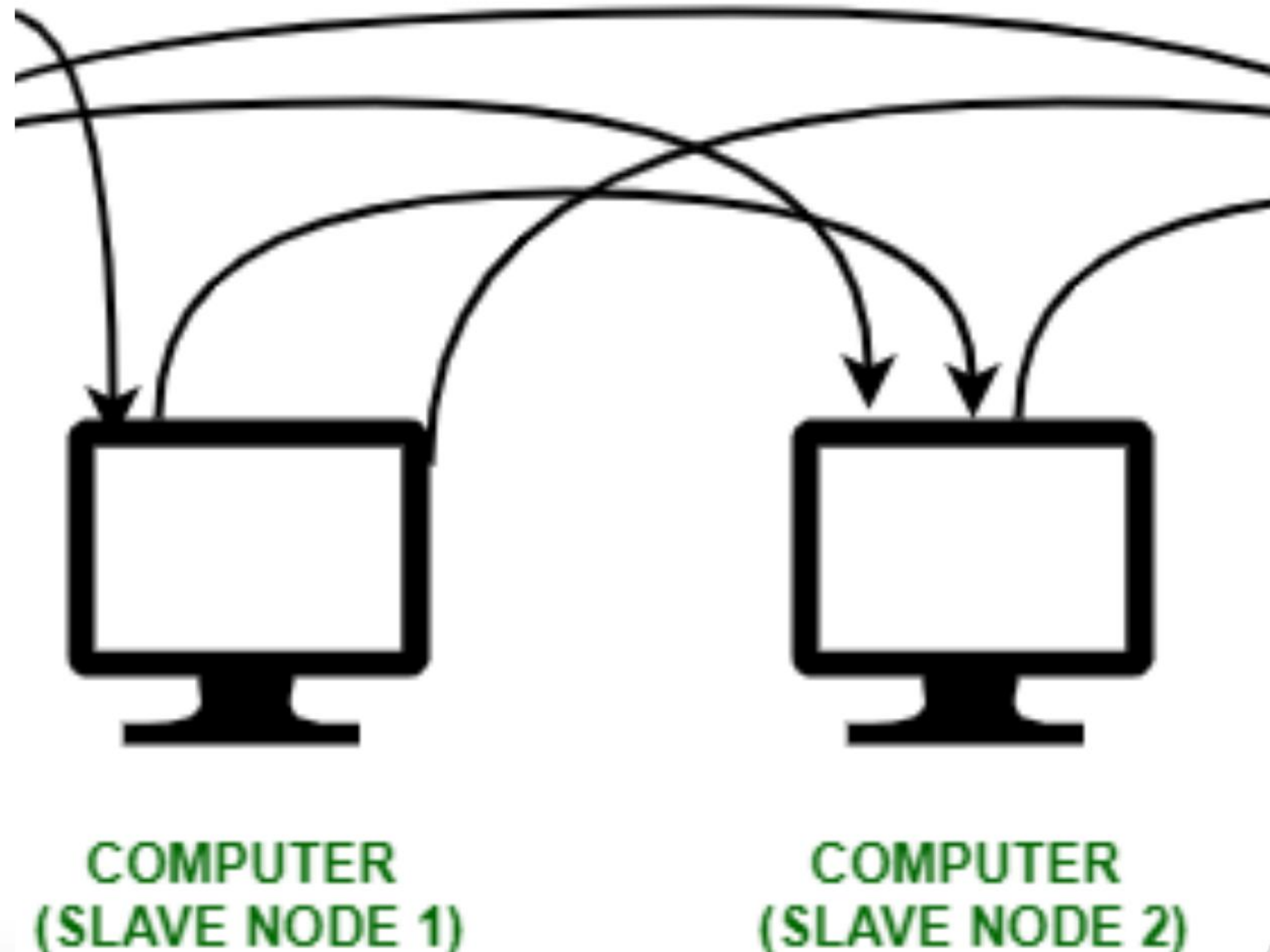
Procesamiento Distribuido Masivo

El Poder de Trabajar en Paralelo

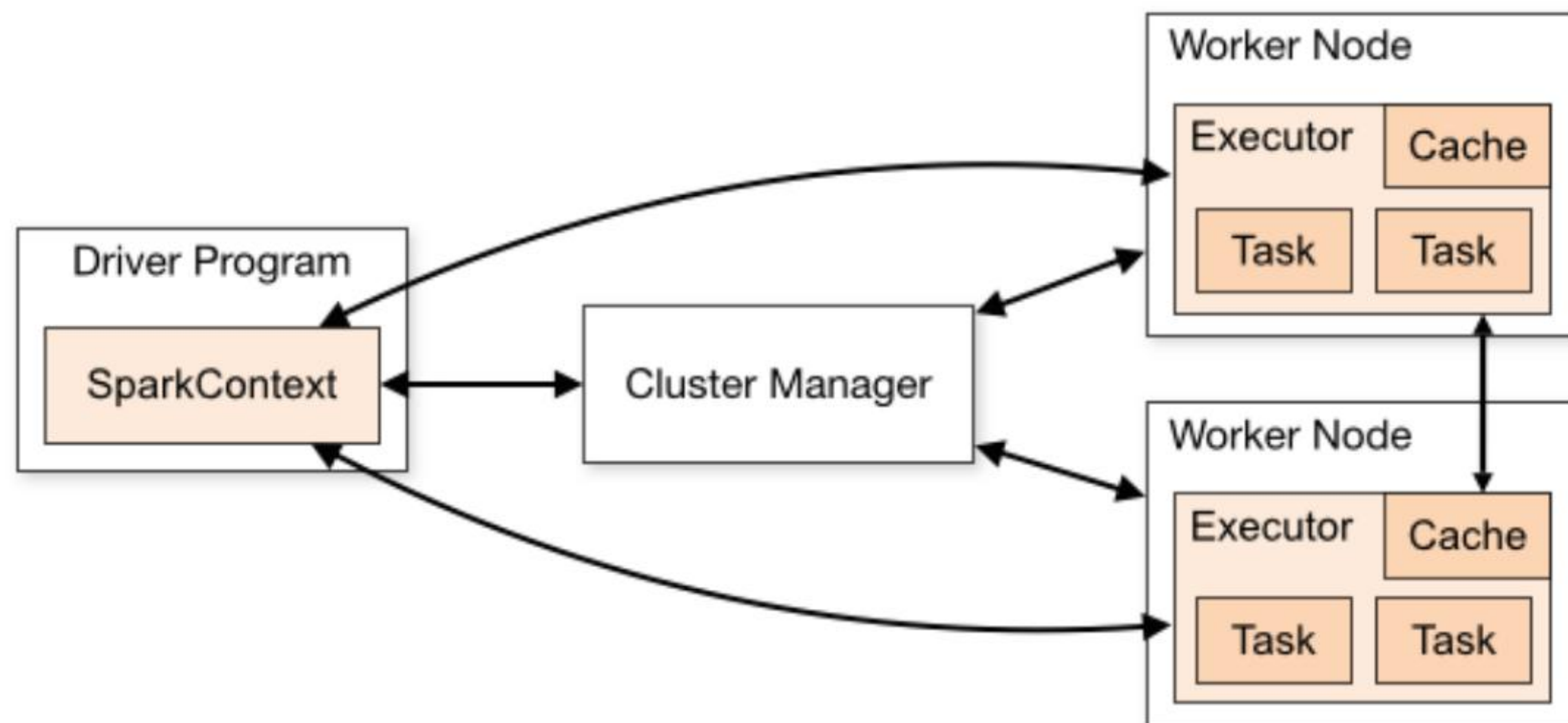
Escenario: Procesar 100 GB de datos en un solo computador requeriría horas debido a limitaciones de RAM y disco.

Solución Distribuida: Dividir los 100 GB entre 10 computadores. Cada máquina procesa únicamente 10 GB de forma simultánea.

Versatilidad de Spark: Funciona con Batch, Streaming, SQL, Machine Learning y Graphos sobre 1 equipo o miles en la nube.



Arquitectura General de Apache Spark



Componentes Principales

-  **Usuario / Código:** Define las transformaciones y acciones sobre los DataFrames.
-  **Driver Node:** El cerebro coordinador de la aplicación.
-  **Cluster Manager:** Administra los recursos físicos del cluster.
-  **Executors:** Procesos worker que ejecutan las tareas pesadas.

Clave Examen: El Driver coordina y planea; los Executors ejecutan las Tasks en memoria/disco.

| El Driver Node: El Cerebro de Spark

Responsabilidades Críticas

-  Ejecuta la función `main()` y mantiene la `SparkSession`.
-  Convierte el código del usuario en un plan de ejecución optimizado (DAG).
-  Solicita recursos al Cluster Manager para crear los Executors.
-  Divide los Jobs en Stages y Tasks, distribuyéndolas a los Executors.
-  Mantiene métricas y expone la interfaz web `Spark UI`.



Analogía: Director de Orquesta

El Director de Orquesta no toca cada instrumento directamente, pero lee la partitura (código), marca el ritmo (DAG) y le indica a cada músico (Executor) cuándo y qué ejecutar.

Peligro OOM: Si ejecutas `.collect()`, envías TODOS los datos distribuidos al Driver. ¡Esto puede tumbar el proceso Driver!

Los Executors: Los Procesos de Trabajo



Procesamiento Activo

Ejecutan las **Tasks** asignadas por el Driver en múltiples hilos (threads) en paralelo sobre las particiones de datos.



Gestión de Memoria

Almacenan datos en memoria RAM (Cache/Persist) y realizan lecturas/escrituras en almacenamiento (Disco/Data Lake/Delta).



Tolerancia a Fallos

Si un Executor muere o falla una máquina, el Driver detecta la pérdida y reasigna sus tareas inmediatamente a otro Executor.

Driver vs. Executors: Comparativa Directa

Característica	Driver Node (El Cerebro)	Executors (Los Obreros)
Cantidad	Exactamente 1 por aplicación Spark.	Múltiples (de 1 a cientos según la escala).
Función Principal	Coordinar, planificar, crear DAG y enviar tareas.	Ejecutar Tasks, procesar datos y escribir en memoria/disco.
Manejo de Datos	Recibe metadata y resultados agregados pequeños.	Procesan las particiones masivas de datos en paralelo.
Riesgo Principal	Out Memory Error por collect() masivo.	Data Skew (desbalanceo) o memoria insuficiente por Task.

"Resumen para el examen: El Driver PIENSA y PLANIFICA. Los Executors TRABAJAN y PROCESAN."

Infraestructura: Cluster & Cluster Managers

¿Qué es un Cluster?

Conjunto de servidores independientes (Worker Nodes) conectados por red que combinan CPU, RAM y disco para funcionar como una sola supercomputadora.

El Rol del Cluster Manager

Componente externo a Spark encargada de asignar y liberar recursos físicos para las aplicaciones.

Tipos de Cluster Managers

Manager	Uso & Contexto
Databricks	Administración 100% automatizada y optimizada.
Kubernetes	Estándar moderno en la nube para contenedores.
YARN	Estándar clásico de ecosistemas Hadoop.
Standalone	Administrador simple integrado nativo en Spark.

| Jerarquía de Ejecución: Job → Stage → Task



1. Job

Trabajo completo desencadenado cuando se invoca una **Acción** (ej: `count()`, `save()`, `display()`).

2. Stage

Subdivisión de un Job. Se crea un nuevo Stage cada vez que se requiere una reorganización de datos (**Shuffle**).

3. Task

Unidad mínima de trabajo enviada a un hilo (core) de un Executor para procesar **1 partición** individual.

| El DAG: Grafo Dirigido Acíclico

¿Qué es el DAG?

Plan de ejecución lógico donde las operaciones son nodos y las dependencias de datos son aristas. Es **dirigido** (vía única) y **acíclico** (no tiene bucles infinitos).

Ventajas de la Optimización por DAG

- ⚡ **Pipelining:** Fusiona múltiples transformaciones en un solo paso.
- ▼ **Predicate Pushdown:** Mueve los filtros al inicio para leer menos datos.
- ✂ **Pruning:** Elimina columnas o pasos innecesarios.

Catalyst Optimizer

El motor Spark SQL toma el DAG y genera planes Lógicos y Físicos optimizados antes de enviar código máquina (Tungsten) a los Executors.

Pregunta de Examen: ¿Quién genera y gestiona el DAG? El **Driver Node**.

| Lazy Evaluation (Evaluación Perezosa)

Spark **NO** ejecuta inmediatamente las transformaciones cuando el desarrollador escribe el código. Únicamente las registra en el DAG.

```
# Nada de esto ejecuta procesamiento real:  
df = spark.read.table("ventas")  
df_filtrado = df.filter(df.monto > 100)  
df_final = df_filtrado.select("cliente", "monto")  
  
# ¡AQUÍ se dispara la ejecución de todo el DAG!  
display(df_final) # ← Acción
```

Beneficios Principales

- 🛡️ Evita ejecuciones e I/O redundantes en disco/red.
- ✂️ Permite al Catalyst Optimizer ver la consulta completa antes de actuar.
- 🧠 Ahorra memoria consumiendo solo los datos finales requeridos.

Narrow vs. Wide Transformations

Narrow Transformations

Cada partición de entrada contribuye a EXACTAMENTE 1 partición de salida.

- No requiere mover datos entre nodos por red.
- Ejecución ultra rápida (Pipelining).
- Ejemplos: `filter()`, `select()`, `withColumn()`, `drop()`.

Wide Transformations

Múltiples particiones contribuyen a crear nuevas particiones de salida.

- Requiere reorganizar datos entre nodos (**Shuffle**).
- Genera un nuevo Stage en el Job.
- Ejemplos: `groupBy()`, `join()`, `distinct()`, `orderBy()`.

| ¿Qué es el Shuffle y por qué es costoso?

Mecanismo del Shuffle

Es la reorganización e intercambio de datos a través de la red entre Executors cuando se necesita agrupar registros por una clave (ej: `groupBy("ciudad")`).

Todos los registros de "Bogotá" deben viajar al mismo Executor para ser sumados o contados.

Estrategias de Optimización: Filtrar antes de agrupar, seleccionar solo columnas necesarias, usar Broadcast Joins en tablas pequeñas.

Costos Asociados al Shuffle

-  **Red:** Transferencia masiva de I/O entre nodos.
-  **Escritura en Disco:** Spill temporal a disco en executors.
-  **Serialización:** Convertir objetos a bytes y viceversa.
-  **Latencia:** Es el mayor cuello de botella en Spark.

Spark UI: La Herramienta de Diagnóstico



Pestañas Jobs & Stages

Muestra el progreso en tiempo real, número de Stages completados, lectura/escritura de Shuffle y tiempo de ejecución por Task.



Pestañas Storage & Executors

Inspecciona DataFrames en cache/memoria y verifica el estado de salud, memoria asignada y GC (Garbage Collection) de los Executors.



Pestaña SQL / Dataframe

Muestra la representación gráfica del DAG detallado, número de filas procesadas por operador y planes lógicos/físicos.

| Demostración en Vivo (Databricks Notebook)

- > Paso 1: Crear un DataFrame con `spark.range(1, 10000000)`.
- ▼ Paso 2: Aplicar transformaciones estrechas (`filter`, `withColumn`) y comprobar en el Notebook que NO hay ejecución.
- ▶ Paso 3: Invocar una acción (`df.count()`) y abrir el Spark UI para analizar el Job de 1 solo Stage.
- ↶ Paso 4: Agregar una transformación ancha (`groupBy`) e inspeccionar cómo surge un **segundo Stage** debido al Shuffle de datos.

| Resumen del Bloque 2 & Próximos Pasos

Conceptos Clave Aprendidos

- ✓ **Driver vs Executor:** Coordinador vs Ejecutores.
- ✓ **Jerarquía:** Job (Acción) → Stage (Shuffle) → Task (Partición).
- ✓ **DAG & Lazy:** Optimización previa a la ejecución.
- ✓ **Transformaciones:** Narrow (Sin Shuffle) vs Wide (Con Shuffle).



Siguiente Bloque

BLOQUE 3: Transformaciones y Acciones en PySpark

Comenzaremos la práctica intensiva de código con selecciones, filtros, agregaciones, funciones con fechas, JSONs y manejo de nulos.

| Bibliografía Recomendada & Recursos

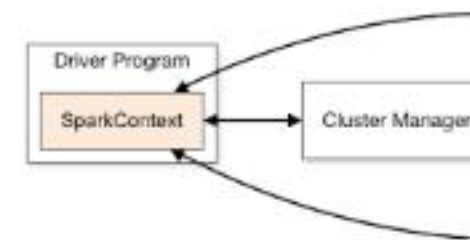
-  **Libros Recomendados:** *"Spark: The Definitive Guide"* (Bill Chambers & Matei Zaharia), *"Learning Spark (2nd Ed)"* (Jules S. Damji et al.), *"High Performance Spark"* (Holden Karau).
-  **Documentación Oficial:** [Apache Spark RDD & Cluster Guide](#) y [Databricks Architecture Docs](#).
-  **Cursos de Certificación:** Databricks Academy — *Databricks Certified Associate Developer for Apache Spark Exam Guide*.

| Image Sources



<https://media.geeksforgeeks.org/wp-content/uploads/20210313225739/UntitledDiagram4.png>

Source: www.geeksforgeeks.org



https://miro.medium.com/1*EzZs4uEuO30IV51KV07_RA.png

Source: medium.com